

How many bytes is your name?

You know "1 byte = 8 bits." But what does that actually mean for the text you use every day?

HoangYell

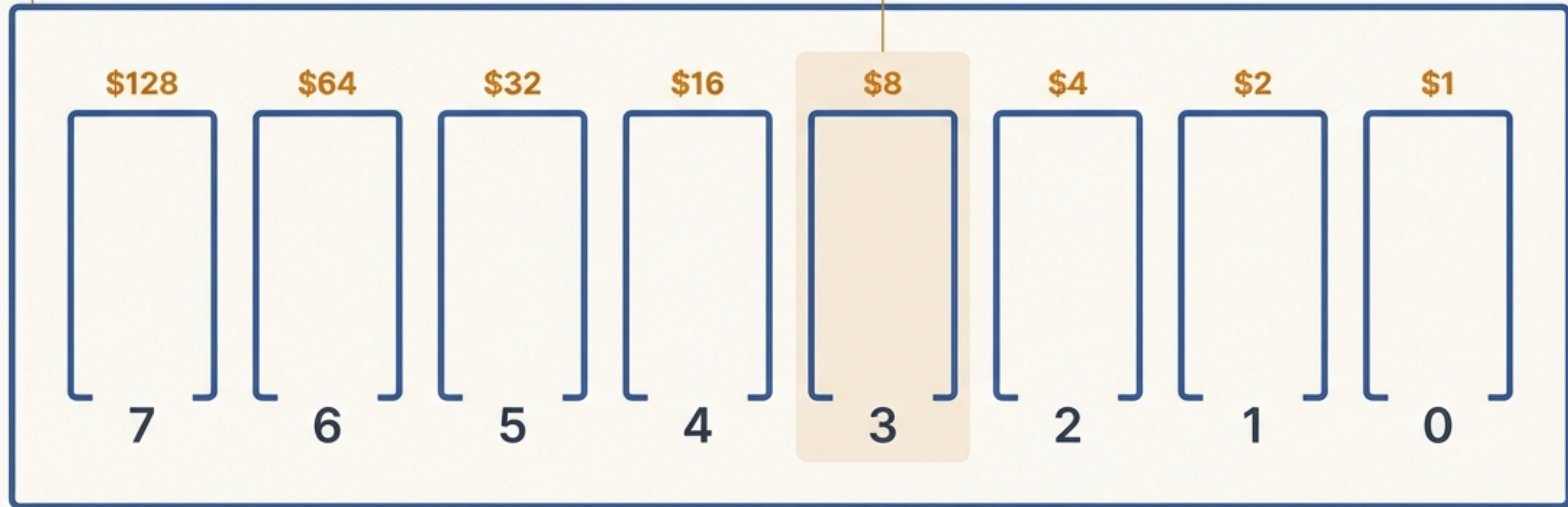


I once spoke with a colleague who could recite "1 byte = 8 bits" in his sleep, yet froze when asked, "How many bytes is your name?" Let's fix that.

Let's Start with a Simple Model: The Parking Lot.

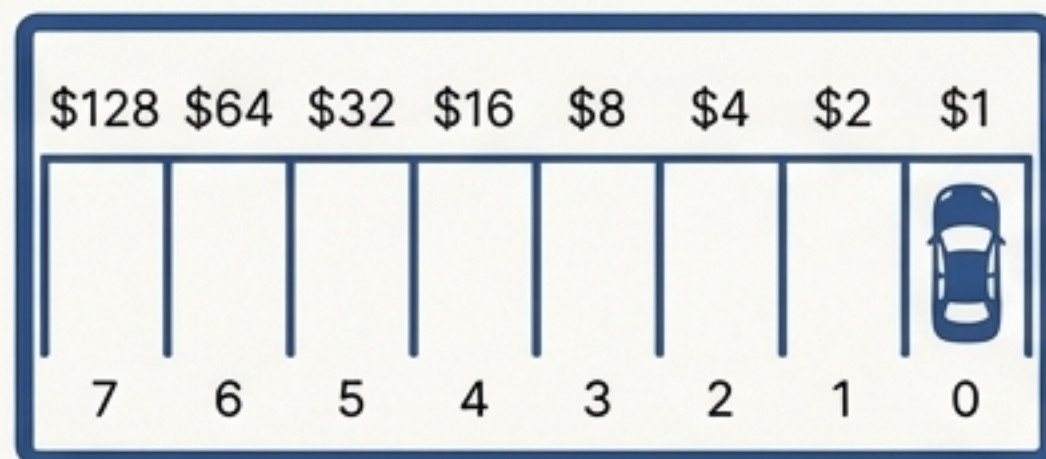
1 byte = The entire parking lot with its 8 slots.

1 bit = A single parking slot. It can be empty ('0') or have a car ('1').



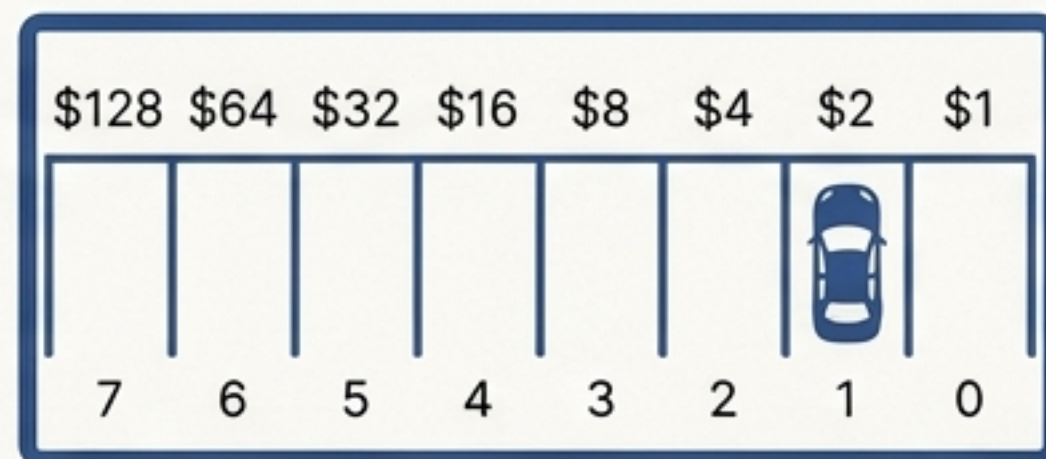
A Byte's Value is the Sum of its Occupied Slots.

The total “revenue” of the parking lot is the sum of the price tags for all occupied slots. With 8 slots, we can generate 256 unique values, from \$0 to \$255.



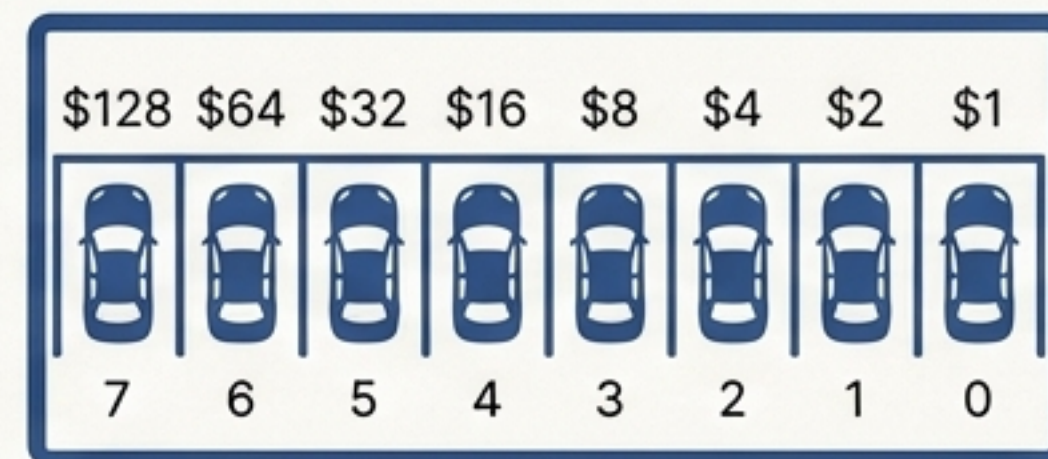
Binary: `00000001`

Revenue: \$1



Binary: `00000010`

Revenue: \$2



Binary: `11111111`

Revenue: \$128 + \$64 + \$32 + \$16
+ \$8 + \$4 + \$2 + \$1 = \$255

The First System: One Parking Lot Per Character.

To store text, computers map characters to numbers. With 256 possible values (0-255), a single byte was the perfect size for every English letter, number, and symbol. This system is called ASCII.



The simple rule of ASCII: **1 Character = 1 Byte.**

In the ASCII World, the Name 'YELL' Takes 4 Bytes.



Each character gets its own dedicated byte.
Simple, clean, and efficient for English.

How We Park the Cars for 'Y' (Decimal 89).

Forget complex math. Just use the "Bit-test method" with this sequence: **128, 64, 32, 16, 8, 4, 2, 1**.

1. Start with 89. Does it fit in 128? **No.** → **Slot 7 is '0'**.

2. Does 89 fit in 64? **Yes.** → **Slot 6 is '1'**.
(Remainder: $89 - 64 = 25$)

3. Does 25 fit in 32? **No.** → **Slot 5 is '0'**.

4. Does 25 fit in 16? **Yes.** → **Slot 4 is '1'**.
(Remainder: $25 - 16 = 9$)

5. Does 9 fit in 8? **Yes.** → **Slot 3 is '1'**.
(Remainder: $9 - 8 = 1$)

6. Does 1 fit in 4? **No.** → **Slot 2 is '0'**.

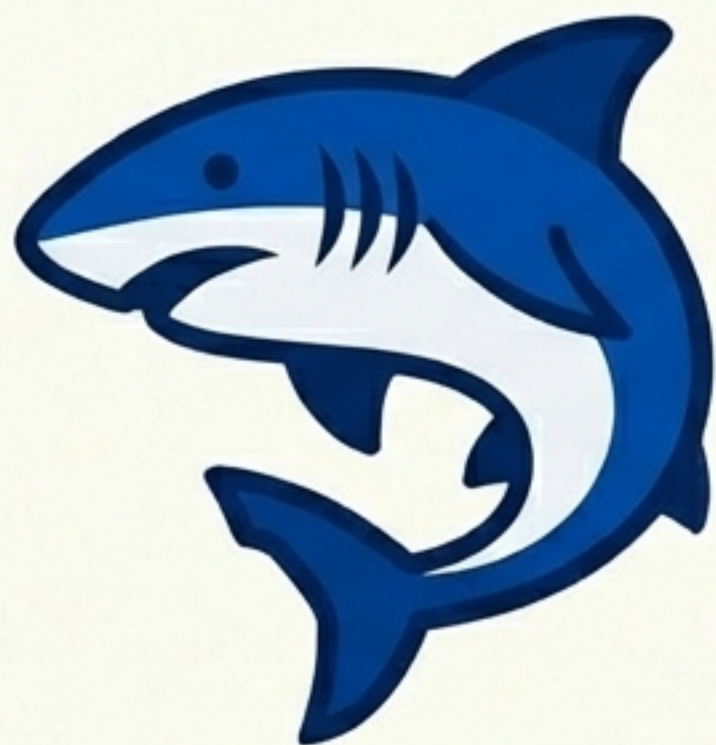
7. Does 1 fit in 2? **No.** → **Slot 1 is '0'**.

8. Does 1 fit in 1? **Yes.** → **Slot 0 is '1'**.



Result: 01011001

But the ASCII Model Breaks for Global Languages.



The number for 🦈 is **129,416**.

Our parking lot's maximum "revenue" is **255**.

ASCII is great for English. But what about 🦈, 你, ô, or ã?
256 slots are not enough. We need a bigger system.

The Solution: A Universal Map and a Flexible System.

Unicode (The Map)



Unicode is a massive character set that assigns a unique number (a 'code point') to every character imaginable. It doesn't say *how** to store them, just what their number is.

🐟 → U+1F988 (129,416)
你 → U+4F60 (20,320)
ô → U+1EC7 (7,879)

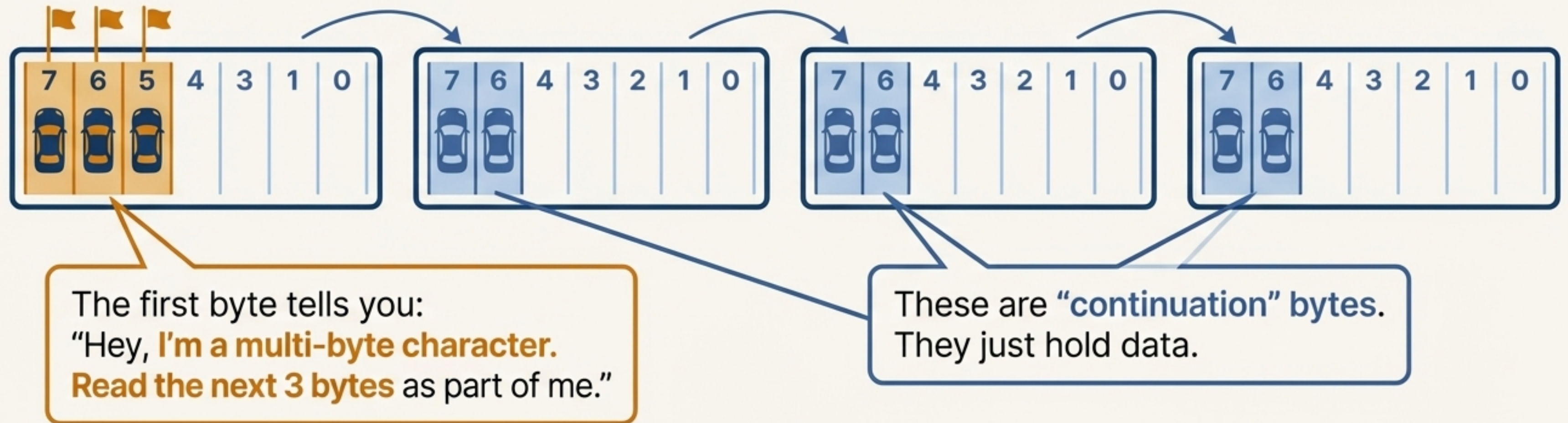
UTF-8 (The Navigation System)



UTF-8 is the encoding that translates a Unicode code point into a sequence of one or more bytes. It's the method for storing any character from the giant map.

UTF-8's Genius is the 'Multi-Lot Chain System'.

How do you know where one character ends and the next begins?
The first few bits of the first byte act as a signal.



The leading bits in UTF-8 solve the chaining problem.

The Byte Templates That Make UTF-8 Work.

Bytes Needed	Template	Notes
1	0 XXXXXXXX	For all standard ASCII characters. Fully backwards-compatible.
2	110 XXXXX 10 XXXXXX	
3	1110 XXXX 10 XXXXXX 10 XXXXXX	
4	11110 XXX 10 XXXXXX 10 XXXXX 10 XXXXXX	

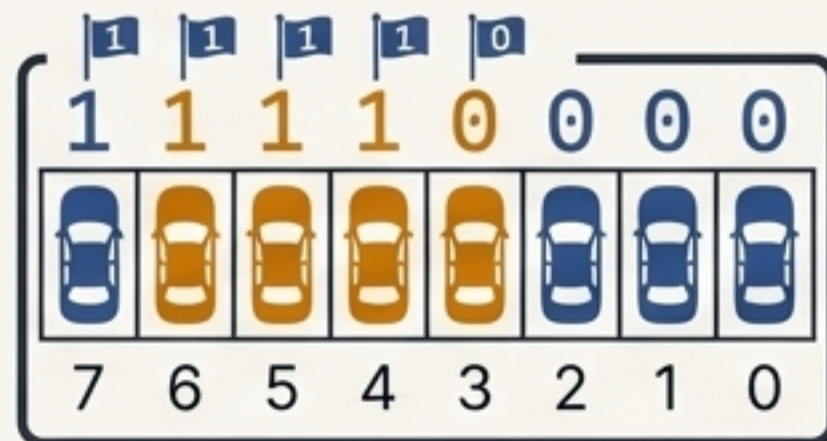
The blue bits are 'header' bits that define the structure. The 'x' bits are the 'payload' where the character's code point data is stored.

Encoding the Shark: A 4-Byte Operation.

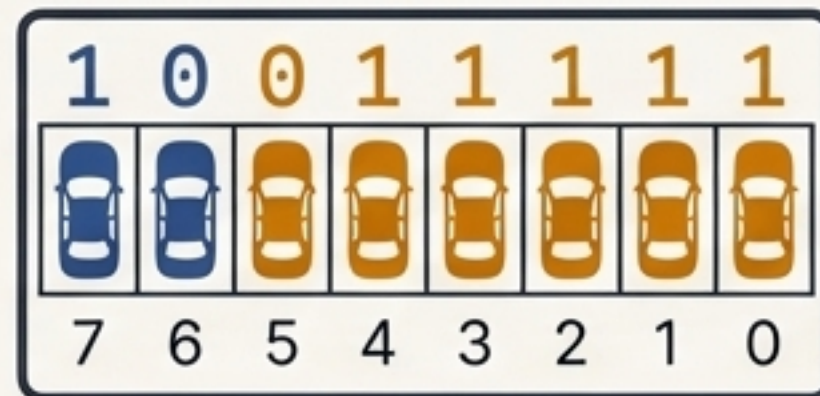
1. **Code Point:** 🦈 = 129,416
2. **Range Check:** This number falls into the 4-byte category.
3. **To Binary:** 129,416 = 1 1111 1001 1000 10000 (**21 bits of data**)
4. **Stuff into Template:**

1 1111 1001 1000 10000

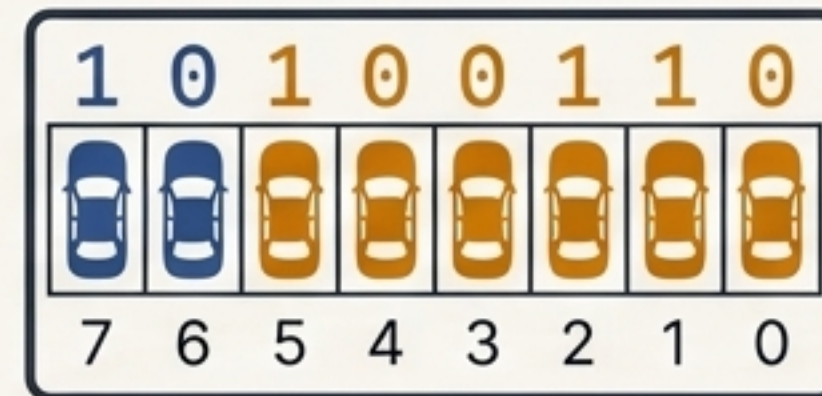
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx



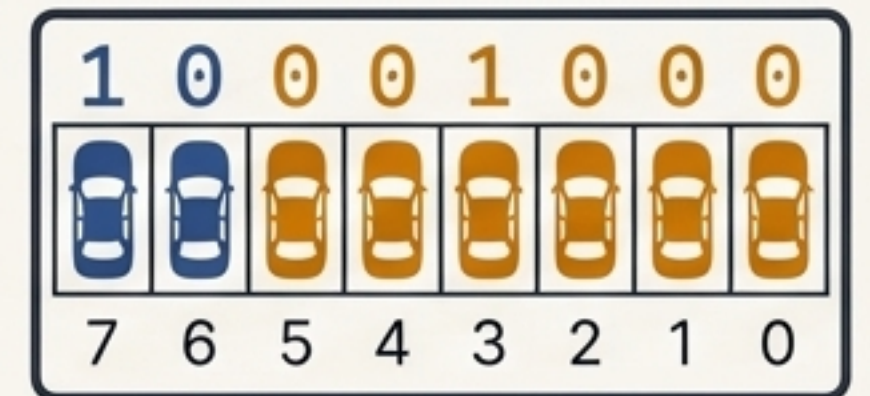
Byte 1: 240



Byte 2: 159



Byte 3: 166



Byte 4: 136

UTF-8 Quick Reference: Size Depends on the Character

Code Point Range	Size	Example Char	Code Point (Hex)	Decimal Value	UTF-8 Bytes (Decimal)
0 – 127	1 byte	Y	U+0059	89	89
128 – 2,047	2 bytes	é	U+00E9	233	195, 169
2,048 – 65,535	3 bytes	你	U+4F60	20,320	228, 189, 160
65,536 – 1,114,111	4 bytes	🐬	U+1F988	129,416	240, 159, 166, 136

UTF-8’s genius is that it’s both efficient (using only 1 byte for ASCII) and expansive (scaling up to 4 bytes for everything else).

So, How Many Bytes is Your Name?

It depends on the characters you use.

Name: "Yell"

$Y(1) + e(1) + l(1) + l(1)$

Total: 4 bytes

Name: "José"

$J(1) + o(1) + s(1) + \text{é}(2)$

Total: 5 bytes

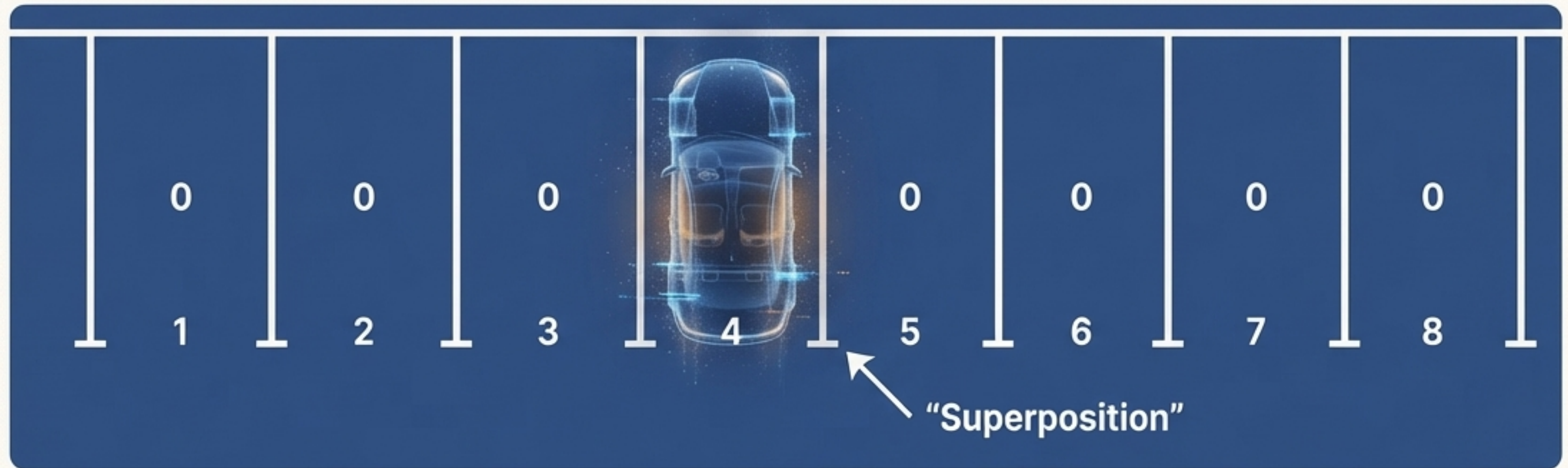
Name: "Hoàng"

$H(1) + o(1) + \text{à}(2) + n(1) + g(1)$

Total: 6 bytes

The Next Step? Breaking the Physics of the Parking Lot.

A classical bit is simple: the slot is either empty (0) or occupied (1).
A quantum bit, or **qubit**, is different.



The car enters a ghostly state: it is **both parked and not parked at the same time**.
Only when you look at it does it collapse into a definite 0 or 1.

It's like Schrödinger's Parking Spot. Spooky? Yes. But it's what allows quantum computers to explore millions of possibilities at once.